TRƯỜNG ĐẠI HỌC KINH TẾ – KỸ THUẬT CÔNG NGHIỆP

KHOA CÔNG NGHỆ THÔNG TIN

HỌC PHẦN: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

(Natural Language Processing)

BÀI TẬP THỰC HÀNH – BÀI 1

INTRODUCTION TO NLP

Giảng viên: Bùi Văn Tân

Khoa Công nghệ Thông tin – UNETI

Hà Nội, 2026

Mục lục

1 Mục tiêu thực hành **2**

2 Bài 1. Khảo sát một corpus văn bản tiếng Việt **2**

3 Bài 2. Tiền xử lý và chuẩn hóa văn bản tiếng Việt **3**

4 Bài 3. Khảo sát nhập nhằng và biến thể ngôn ngữ **4**

5 Bài 4. Phân loại văn bản với TF–IDF và Logistic Regression **6**

6 Bài 5. Mất cân bằng dữ liệu và lựa chọn metric **7**

7 Bài 6. Trích xuất thực thể bằng luật đơn giản **8**

8 Bài 7. Information Retrieval với TF–IDF **9**

9 Bài 8. Đo tương đồng giữa các câu **10**

10 Bài 9. Mô hình ngôn ngữ Bigram và sinh văn bản đơn giản **11**

11 Bài 10. Xây dựng NLP Pipeline và thực hiện Error Analysis **12**

12 Yêu cầu báo cáo thực hành **14**

13 Câu hỏi thảo luận sau thực hành **14**

1. Mục tiêu thực hành

Tài liệu thực hành này được xây dựng tương ứng với nội dung của Bài 1: Introduction to NLP. Các bài tập không đi sâu thay thế cho các bài lý thuyết sau về N-gram, word embeddings, RNN/LSTM, Transformer hay LLM, mà giúp sinh viên hình thành tư duy tổng quan về dữ liệu ngôn ngữ, bài toán NLP, NLP pipeline, biểu diễn văn bản, mô hình hóa, đánh giá và phân tích lỗi.

Yêu cầu cần đạt

Sau khi hoàn thành 10 bài tập, sinh viên có thể:

- khảo sát và mô tả một tập dữ liệu văn bản;
- thực hiện một số bước tiền xử lý văn bản tiếng Việt cơ bản;
- nhận biết tính nhập nhằng, biến thể và phụ thuộc ngữ cảnh của ngôn ngữ;
- xây dựng các mô hình NLP cơ bản cho phân loại, trích xuất, truy hồi và đo tương đồng;
- lựa chọn chỉ số đánh giá phù hợp và thực hiện phân tích lỗi;
- ghép các thành phần thành một NLP pipeline hoàn chỉnh;
- nhận biết sự chuyển dịch từ phương pháp dựa trên luật sang phương pháp học từ dữ liệu.

Môi trường đề xuất: Python 3.10+, Jupyter Notebook/Google Colab; các thư viện `numpy`, `pandas`, `scikit-learn`, `matplotlib`. Các bài tập được thiết kế để không bắt buộc GPU.

2. Bài 1. Khảo sát một corpus văn bản tiếng Việt

Đề bài

Cho một tập các câu phản hồi của sinh viên. Viết chương trình thống kê: số văn bản, số token của từng văn bản, độ dài trung bình, từ xuất hiện nhiều nhất và phân bố độ dài. Qua đó, nhận xét vì sao cần *Exploratory Data Analysis – EDA* trước khi xây dựng mô hình NLP.

Hướng dẫn làm bài

1. Tạo một corpus nhỏ gồm các phản hồi tích cực, trung tính và tiêu cực.
2. Chuẩn hóa chữ thường và tách token ở mức đơn giản bằng khoảng trắng.
3. Sử dụng `Counter` để đếm tần suất token.
4. Tính số token trung bình và xác định câu dài nhất/ngắn nhất.
5. Vẽ histogram độ dài văn bản và nhận xét dữ liệu.

Code Python tham khảo

```
from collections import Counter
import matplotlib.pyplot as plt

texts = [
    "Bài giảng rất dễ hiểu và có nhiều ví dụ",
    "Nội dung môn học khá thú vị",
    "Bài tập hơi khó nhưng hữu ích",
    "Tài liệu cần bổ sung thêm ví dụ thực hành",
    "Giảng viên giải thích rõ ràng",
    "Một số phần trình bày còn nhanh"
]

tokenized = [text.lower().split() for text in texts]
lengths = [len(tokens) for tokens in tokenized]
all_tokens = [token for sent in tokenized for token in sent]
freq = Counter(all_tokens)

print("Số văn bản:", len(texts))
print("Độ dài trung bình:", sum(lengths) / len(lengths))
print("10 token phổ biến:", freq.most_common(10))
print("Câu dài nhất:", texts[lengths.index(max(lengths))])
print("Câu ngắn nhất:", texts[lengths.index(min(lengths))])

plt.hist(lengths, bins=range(min(lengths), max(lengths) + 2),
 ↪  edgecolor="black")
plt.xlabel("Số token")
plt.ylabel("Số văn bản")
plt.title("Phân bố độ dài văn bản")
plt.show()
```

3. Bài 2. Tiền xử lý và chuẩn hóa văn bản tiếng Việt

Đề bài

Xây dựng hàm tiền xử lý cho các phản hồi tiếng Việt có URL, khoảng trắng dư thừa, chữ hoa/chữ thường và dấu câu. So sánh văn bản trước và sau xử lý. Không loại bỏ thông tin một cách máy móc: sinh viên phải giải thích thành phần nào nên giữ lại tùy bài toán.

Hướng dẫn làm bài

1. Chuẩn hóa Unicode bằng `unicodedata.normalize`.
2. Chuyển về chữ thường.
3. Thay URL bằng token `<URL>` thay vì xóa hoàn toàn.

4. Chuẩn hóa khoảng trắng và giảm dấu câu lặp quá nhiều.
5. Thảo luận: emoji, hashtag và dấu chấm than có nên xóa trong bài toán sentiment hay không?

Code Python tham khảo

```
import re
import unicodedata

def preprocess(text):
    text = unicodedata.normalize("NFC", text)
    text = text.lower().strip()
    text = re.sub(r"https?://\S+|www\.\S+", " <URL> ", text)
    text = re.sub(r"([!?.,])\1+", r"\1", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

samples = [
    "  Sản phẩm RẤT tốt!!! Xem tại https://example.com  ",
    "Giao hàng chậm.... nhưng hỗ trợ ổn!!",
    "Môn NLP thú vị!!! #NLP"
]

for s in samples:
    print("RAW :", s)
    print("CLEAN:", preprocess(s))
    print("-" * 60)
```

Lưu ý

Tiền xử lý phải phụ thuộc vào bài toán và mô hình. Với các mô hình tiền huấn luyện, không nên tùy tiện stemming, xóa stopwords hoặc thay đổi văn bản nếu việc đó làm mất thông tin mà tokenizer/mô hình có thể khai thác.

4. Bài 3. Khảo sát nhập nhằng và biến thể ngôn ngữ

Đề bài

Viết chương trình minh họa hai thách thức được trình bày trong Bài 1: ambiguity và language variation. Với một danh sách các cách viết không chuẩn, hãy chuẩn hóa chúng bằng từ điển ánh xạ; đồng thời xây dựng một ví dụ cho thấy luật dựa trên từ khóa có thể thất bại khi câu phụ thuộc vào ngữ cảnh.

Hướng dẫn làm bài

1. Xây dựng từ điển chuẩn hóa một số biến thể như ko, k, đc, sv.
2. Viết hàm thay thế theo token.
3. Xây dựng bộ phân loại cảm xúc dựa trên một vài từ khóa.
4. Kiểm thử với câu phủ định như “không tốt” và câu có hai vế trái chiều.
5. Giải thích vì sao luật từ khóa không đủ để biểu diễn semantics/pragmatics.

Code Python tham khảo

```
NORMALIZE = {
    "ko": "không",
    "k": "không",
    "đc": "được",
    "sv": "sinh_viên",
    "gv": "giảng_viên"
}

def normalize_variants(text):
    tokens = text.lower().split()
    return " ".join(NORMALIZE.get(t, t) for t in tokens)

def keyword_sentiment(text):
    text = text.lower()
    positive = ["tốt", "hay", "thú vị", "hữu ích"]
    negative = ["tệ", "chậm", "khó hiểu"]
    score = sum(w in text for w in positive)
    score -= sum(w in text for w in negative)
    return "positive" if score > 0 else "negative" if score < 0 else
    ↪  "neutral"

examples = [
    "môn này ko khó",
    "bài giảng không tốt",
    "nội dung hay nhưng trình bày hơi khó hiểu"
]

for x in examples:
    x_norm = normalize_variants(x)
    print(x, "->", x_norm, "->", keyword_sentiment(x_norm))
```

5. Bài 4. Phân loại văn bản với TF–IDF và Logistic Regression

Đề bài

Xây dựng một hệ thống *Text Classification* đơn giản để phân loại phản hồi thành `positive` và `negative`. Đây là bài tập minh họa pipeline: dữ liệu → biểu diễn văn bản → mô hình → dự đoán → đánh giá.

Hướng dẫn làm bài

1. Tạo hoặc thay thế bằng tập dữ liệu có tối thiểu 100 mẫu khi làm báo cáo chính thức.
2. Chia train/test bằng `train_test_split`.
3. Dùng `TfidfVectorizer` để biểu diễn văn bản.
4. Huấn luyện `LogisticRegression`.
5. Đánh giá bằng Accuracy, Precision, Recall và F1-score.

Code Python tham khảo

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

texts = [
    "sản phẩm rất tốt", "dịch vụ tuyệt vời",
    "tôi rất hài lòng", "giao hàng nhanh",
    "chất lượng tốt", "trải nghiệm rất hay",
    "sản phẩm quá tệ", "dịch vụ rất chậm",
    "tôi không hài lòng", "chất lượng kém",
    "giao hàng quá chậm", "trải nghiệm tệ"
]
labels = ["positive"] * 6 + ["negative"] * 6

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.33, random_state=42, stratify=labels
)

model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)
```

```
print(classification_report(y_test, pred, zero_division=0))

new_texts = ["sản phẩm tốt nhưng giao hàng chậm"]
print(model.predict(new_texts))
```

6. Bài 5. Mất cân bằng dữ liệu và lựa chọn metric

Đề bài

Mô phỏng một tập sentiment mất cân bằng và chứng minh rằng Accuracy cao chưa chắc mô hình tốt. So sánh Accuracy với Precision, Recall, F1-score và confusion matrix.

Hướng dẫn làm bài

1. Tạo 100 nhãn thật, trong đó 90 mẫu positive và 10 mẫu negative.
2. Giả sử mô hình dự đoán tất cả là positive.
3. Tính Accuracy và báo cáo phân loại.
4. Quan sát Recall/F1 của lớp negative.
5. Giải thích metric nào phù hợp hơn khi lớp thiểu số quan trọng.

Code Python tham khảo

```
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

# 90 positive, 10 negative
y_true = ["positive"] * 90 + ["negative"] * 10

# Mô hình ngây thơ: luôn dự đoán positive
y_pred = ["positive"] * 100

print("Accuracy:", accuracy_score(y_true, y_pred))
print(classification_report(y_true, y_pred, zero_division=0))

cm = confusion_matrix(y_true, y_pred,
                      labels=["positive", "negative"])
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["positive", "negative"]
)
disp.plot()
```

```
plt.show()
```

7. Bài 6. Trích xuất thực thể bằng luật đơn giản

Đề bài

Xây dựng một hệ thống *Information Extraction* đơn giản để phát hiện email, số điện thoại và ngày tháng trong văn bản. Bài tập minh họa cách tiếp cận rule-based NLP và giới hạn của nó.

Hướng dẫn làm bài

1. Sử dụng regular expressions để xây dựng ba mẫu: email, số điện thoại, ngày tháng.
2. Chạy trên nhiều câu khác nhau.
3. Bổ sung một định dạng mới và quan sát việc phải sửa luật thủ công.
4. Phân tích ưu điểm: dễ hiểu, dễ kiểm soát; hạn chế: khó bao phủ mọi biến thể.

Code Python tham khảo

```
import re

text = """
Liên hệ qua email student@uneti.edu.vn hoặc sô´0912345678.
Lịch bảo vệ dự kiến ngày 15/10/2026.
"""

patterns = {
    "EMAIL": r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}",
    "PHONE": r"\b0\d{9}\b",
    "DATE": r"\b\d{1,2}/\d{1,2}/\d{4}\b"
}

for entity_type, pattern in patterns.items():
    values = re.findall(pattern, text)
    print(entity_type, values)
```

Lưu ý

Đây chưa phải là một hệ thống NER tổng quát. Mục đích của bài là giúp sinh viên phân biệt *rule-based extraction* với *learned sequence labeling/NER*.

8. Bài 7. Information Retrieval với TF–IDF

Đề bài

Xây dựng một công cụ tìm kiếm văn bản nhỏ. Cho một tập tài liệu và một truy vấn, hệ thống phải xếp hạng tài liệu theo cosine similarity giữa vector TF–IDF của truy vấn và tài liệu.

Hướng dẫn làm bài

1. Tạo một corpus gồm 5–10 đoạn văn thuộc các chủ đề khác nhau.
2. Biểu diễn corpus bằng `TfidfVectorizer`.
3. Biểu diễn truy vấn trong cùng không gian vector.
4. Tính cosine similarity và sắp xếp giảm dần.
5. Thử truy vấn dùng từ khác nhưng cùng nghĩa và nhận xét hạn chế của lexical retrieval.

Code Python tham khảo

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

corpus = [
    "NLP nghiên cứu cách máy tính xử lý ngôn ngữ tự nhiên",
    "Học máy cho phép mô hình học quy luật từ dữ liệu",
    "Học sâu sử dụng mạng nơ ron nhiều tầng",
    "Transformer sử dụng cơ chế attention để mô hình hóa ngữ cảnh",
    "Truy hồi thông tin tìm các tài liệu liên quan đến truy vấn"
]

query = "mô hình xử lý ngôn ngữ bằng attention"

vectorizer = TfidfVectorizer()
doc_vectors = vectorizer.fit_transform(corpus)
query_vector = vectorizer.transform([query])

scores = cosine_similarity(query_vector, doc_vectors)[0]
ranking = np.argsort(scores)[::-1]

for rank, idx in enumerate(ranking, start=1):
    print(rank, round(scores[idx], 4), corpus[idx])
```

9. Bài 8. Đo tương đồng giữa các câu

Đề bài

So sánh hai cách đo độ giống nhau giữa các câu: Jaccard similarity trên tập token và cosine similarity trên TF–IDF. Phân tích các trường hợp hai câu gần nghĩa nhưng ít từ chung.

Hướng dẫn làm bài

1. Cài đặt Jaccard similarity.
2. Tính TF–IDF cosine similarity cho cùng các cặp câu.
3. So sánh ít nhất ba cặp: gần nghĩa, khác nghĩa, có nhiều từ chung nhưng ý nghĩa khác.
4. Nhận xét vì sao lexical similarity chưa đồng nhất với semantic similarity.

Code Python tham khảo

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def jaccard(s1, s2):
    a = set(s1.lower().split())
    b = set(s2.lower().split())
    return len(a & b) / len(a | b) if a | b else 0.0

pairs = [
    ("sinh viên đang học NLP", "sinh viên học xử lý ngôn ngữ tự
    ↪  nhiên"),
    ("môn học này rất thú vị", "bài giảng này rất hay"),
    ("sản phẩm không tốt", "sản phẩm rất tốt")
]

for s1, s2 in pairs:
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform([s1, s2])
    cosine = cosine_similarity(X[0], X[1])[0, 0]
    print("\nS1:", s1)
    print("S2:", s2)
    print("Jaccard:", round(jaccard(s1, s2), 4))
    print("TF-IDF cosine:", round(cosine, 4))
```

10. Bài 9. Mô hình ngôn ngữ Bigram và sinh văn bản đơn giản

Đề bài

Xây dựng một *bigram language model* đơn giản từ corpus nhỏ, sau đó sinh một chuỗi token dựa trên xác suất chuyển tiếp. Bài tập nhằm minh họa giai đoạn Statistical NLP và khái niệm language modeling trước khi học sâu ở các bài sau.

Hướng dẫn làm bài

1. Thêm token <s> và </s> vào mỗi câu.
2. Đếm số lần xuất hiện của cặp (w_i, w_{i+1}).
3. Từ một token hiện tại, lấy danh sách token có thể xuất hiện tiếp theo.
4. Lấy mẫu theo tần suất để sinh câu.
5. Chạy nhiều lần và nhận xét tính cục bộ của mô hình bigram.

Code Python tham khảo

```
from collections import defaultdict, Counter
import random

sentences = [
    "sinh viên học xử lý ngôn ngữ tự nhiên",
    "sinh viên học học máy",
    "xử lý ngôn ngữ tự nhiên là lĩnh vực thú vị",
    "học máy được ứng dụng trong xử lý ngôn ngữ tự nhiên"
]

next_words = defaultdict(Counter)

for sentence in sentences:
    tokens = ["<s>"] + sentence.lower().split() + ["</s>"]
    for w1, w2 in zip(tokens[:-1], tokens[1:]):
        next_words[w1][w2] += 1

def sample_next(counter):
    words = list(counter.keys())
    weights = list(counter.values())
    return random.choices(words, weights=weights, k=1)[0]

def generate(max_len=20):
    current = "<s>"
    output = []
    for _ in range(max_len):
        if current not in next_words:
```

```
            break
        current = sample_next(next_words[current])
        if current == "</s>":
            break
        output.append(current)
    return " ".join(output)

for _ in range(5):
    print(generate())
```

11. Bài 10. Xây dựng NLP Pipeline và thực hiện Error Analysis

Đề bài

Xây dựng một pipeline hoàn chỉnh cho bài toán phân loại phản hồi. Sau khi huấn luyện, chương trình phải tự động liệt kê các mẫu dự đoán sai để sinh viên thực hiện error analysis. Đây là bài tập tổng hợp của Bài 1.

Hướng dẫn làm bài

1. Chuẩn bị tập dữ liệu gồm văn bản và nhãn.
2. Chia dữ liệu train/test, tránh để cùng một văn bản xuất hiện ở cả hai tập.
3. Xây dựng pipeline `TF-IDF + Logistic Regression`.
4. Tính Accuracy và F1-score.
5. Liệt kê các trường hợp dự đoán sai.
6. Phân loại nguyên nhân lỗi: phủ định, từ mới, câu nhiều vế, thiếu ngữ cảnh, nhãn nhập nhằng, dữ liệu quá ít, v.v.
7. Đề xuất ít nhất hai cách cải tiến dữ liệu hoặc mô hình.

Code Python tham khảo

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

# Dữ liệu minh họa; khi làm bài cần mở rộng tập dữ liệu.
data = [
    ("bài giảng rất dễ hiểu", "positive"),
```

```
    ("nội dung rất hữu ích", "positive"),
    ("ví dụ minh họa rất hay", "positive"),
    ("tài liệu được trình bày rõ ràng", "positive"),
    ("bài tập giúp tôi hiểu bài", "positive"),
    ("môn học khá thú vị", "positive"),
    ("bài giảng khó hiểu", "negative"),
    ("nội dung trình bày quá nhanh", "negative"),
    ("tài liệu còn thiếu ví dụ", "negative"),
    ("bài tập quá khó", "negative"),
    ("tôi không hiểu phần này", "negative"),
    ("nội dung chưa rõ ràng", "negative")
]

df = pd.DataFrame(data, columns=["text", "label"])
X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["label"], test_size=0.33,
    random_state=7, stratify=df["label"]
)

pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=1000))
])

pipeline.fit(X_train, y_train)
pred = pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Macro-F1:", f1_score(y_test, pred, average="macro"))

result = pd.DataFrame({
    "text": X_test.values,
    "true": y_test.values,
    "pred": pred
})

errors = result[result["true"] != result["pred"]]
print("\nCác trường hợp dự đoán sai:")
print(errors.to_string(index=False))
```

Yêu cầu mở rộng

Sinh viên không dừng ở việc chạy code mẫu. Hãy thay tập dữ liệu minh họa bằng tối thiểu 100–200 phản hồi, thực hiện EDA, huấn luyện mô hình, báo cáo metric và chọn ít nhất 5 trường hợp lỗi để phân tích nguyên nhân. Nếu mô hình không tạo đủ 5 lỗi, hãy bổ sung các câu kiểm thử khó do sinh viên tự thiết kế.

12. Yêu cầu báo cáo thực hành

Mỗi nhóm/cá nhân nộp một notebook và một báo cáo ngắn. Báo cáo nên gồm:

1. mô tả bài toán và dữ liệu;
2. các bước xử lý đã thực hiện;
3. kết quả chạy chương trình;
4. bảng hoặc hình minh họa khi phù hợp;
5. nhận xét về kết quả và các trường hợp lỗi;
6. kết luận: bài tập minh họa thành phần nào của NLP pipeline hoặc giai đoạn nào trong sự phát triển của NLP.

Lưu ý

Các đoạn code trong tài liệu là mã tham khảo tối thiểu. Sinh viên được khuyến khích mở rộng dữ liệu, cải tiến cách tiền xử lý, thử mô hình khác và giải thích sự khác biệt. Mục tiêu của thực hành không chỉ là tạo ra chương trình chạy được mà còn phải hiểu vì sao lựa chọn một phương pháp và kết quả cho thấy điều gì về bài toán NLP.

13. Câu hỏi thảo luận sau thực hành

1. Vì sao cùng một bước tiền xử lý có thể hữu ích cho mô hình này nhưng gây mất thông tin đối với mô hình khác?
2. Vì sao lexical similarity không đồng nhất với semantic similarity?
3. Khi nào hệ thống dựa trên luật phù hợp hơn một mô hình học máy?
4. Vì sao Accuracy không đủ để đánh giá dữ liệu mất cân bằng?
5. Trong 10 bài tập, những bước nào tương ứng với Data Collection, Preprocessing, EDA, Representation, Modeling và Evaluation?
6. Những hạn chế nào của các phương pháp trong tài liệu có thể được giải quyết tốt hơn bằng word embeddings, Transformer hoặc pre-trained language models?

— HẾT —